

Arquitectura del Sistema

El proyecto está construido como una **Single Page Application (SPA)** moderna, diseñada para ser ligera, rápida y fácil de desplegar, eliminando la necesidad de un backend complejo para esta versión de evaluación.

Stack Tecnológico

- **Frontend Framework:** React 18 (con TypeScript)
- **Build Tool:** Vite (para un desarrollo y compilación ultrarrápidos)
- **Estilos:** Tailwind CSS (Utility-first CSS framework)
- **Iconografía:** Lucide React

Diagrama de Arquitectura (Conceptual)

```
graph TD
    User[Usuario (Evaluador/Candidato)] --> Client[Cliente React (SPA)]

    subgraph "Frontend Layer"
        Client --> Router[Gestor de Vistas (State-based)]
        Router --> Modules[Módulos (Dashboard, Examen, Resultados)]
        Modules --> Services[Capa de Servicios]
    end

    subgraph "Data Layer (Local)"
        Services --> LocalStorage[(LocalStorage / Persistencia)]
    end

    subgraph "External AI Layer"
        Services --> AI_Adapter[AI Service Adapter]
        AI_Adapter --> LLM[Modelo de IA (LLM Externo)]
    end
```

Componentes Clave

1. Gestión de Estado (State Management)

La aplicación utiliza el estado local de React (`useState`, `useCallback`) combinado con `useEffect` para la persistencia. No se utiliza Redux ni Context API complejo debido a la naturaleza autocontenida de la prueba.

2. Persistencia de Datos (Data Layer)

Para facilitar la evaluación y portabilidad sin configurar bases de datos externas, el sistema utiliza **LocalStorage** como base de datos principal.

- * `tbl_assessments`: Almacena las definiciones de las pruebas (JSON).
- * `tbl_candidates`: Almacena los perfiles, estados y resultados de los candidatos.

* `TBL_API_KEYS`: Almacena de forma segura (localmente) las claves de acceso a los modelos de IA.

3. Capa de Servicios de IA

El archivo `geminiService.ts` actúa como un **Facade Pattern**. * Abstrae la complejidad de las llamadas a la API del modelo de IA. * Gestiona la construcción de Prompts (Ingeniería de Prompts). * Fuerza la salida en formato JSON estructurado (JSON Mode) para garantizar que la aplicación pueda procesar las respuestas programáticamente. * Implementa lógica de “Fallback” o selección de proveedor si fuera necesario.

Seguridad

- **Client-Side:** Al ser una demo técnica/evaluación que corre en el cliente, las API Keys se guardan en el navegador del usuario. En un entorno de producción real, esto se movería a un Backend Proxy.
- **Validación:** Los inputs del candidato son sanitizados y evaluados por la IA en busca de inyecciones de código o malas prácticas de seguridad.